续表

栈	输入	动作
$E'T'$**id**	**id**+**id** $	输出 $F\rightarrow$**id**
$E'T'$	+**id** $	匹配 **id**
E'	+**id** $	输出 $T'\rightarrow\varepsilon$
$E'T$+	+**id** $	输出 $E'\rightarrow$+TE'
$E'T$	**id** $	匹配+
$E'TF'$	**id** $	输出 $T\rightarrow FT'$
$E'T'$**id**	**id** $	输出 $F\rightarrow$**id**
$E'T'$	$	匹配 **id**
$E'	$	输出 $T'\rightarrow\varepsilon$
$	$	输出 $E'\rightarrow\varepsilon$

3.3.5 构造预测分析表

对于非递归的预测分析来说,剩下的问题是如何构造预测分析表。下面的算法为文法 G 构造预测分析表 $M[A, a]$,其中 A 是非终结符,a 是终结符或 $。这个算法的思想如下:如果 $A\rightarrow\alpha$ 是产生式且 a 在 $FIRST(\alpha)$ 中,那么在当前输入符号为 a 时,分析器选择用 α 展开 A。唯一的复杂情况是 $\alpha\Rightarrow^{*}\varepsilon$,在这种情况下,如果当前输入符号(包括 $)在 $FOLLOW(A)$ 中,仍应用 α 展开 A。

算法 3.2 构造预测分析表。

输入 文法 G。

输出 分析表 M。

方法 对文法的每个产生式 $A\rightarrow\alpha$,执行(1)和(2)。

(1) 对 $FIRST(\alpha)$ 的每个终结符 a,把 $A\rightarrow\alpha$ 加入 $M[A, a]$。

(2) 如果 ε 在 $FIRST(\alpha)$ 中,对 $FOLLOW(A)$ 的每个终结符 b(包括 $),把 $A\rightarrow\alpha$ 加入 $M[A, b]$(包括 $M[A, \$]$)。

M 剩下的条目没有定义,都是出错条目,通常用空白表示。 □

例 3.14 把算法3.2用于文法(3.8),产生的分析表见表3.1。因为 $FIRST(TE')=FIRST(T)=\{($, **id**$\}$,因此产生式 $E\rightarrow TE'$ 加入 $M[E, (]$和 $M[E,$ **id**$]$。

产生式 $E'\rightarrow$ +TE'加入 $M[E', +]$是显然的。因为 $\varepsilon\in FIRST(E')$,并且 $FOLLOW(E')=\{), \$\}$,因此产生式 $E'\rightarrow\varepsilon$ 加入 $M[E',)]$和 $M[E', \$]$。 □

算法 3.2 可用于任何文法 G 来产生分析表 M。然而对某些文法，M 可能含有一些多重定义的条目。例如 G 是左递归或二义的，则 M 至少含一个多重定义的条目。

例 3.15 以例 3.7 的条件语句为例，为方便起见，将文法重写如下：

$$\begin{aligned} &stmt \rightarrow \textbf{if}\ expr\ \textbf{then}\ stmt\ e_part \mid \textbf{other} \quad /* \quad e_part\ 指\ optional_else_part \quad */ \\ &e_part \rightarrow \textbf{else}\ stmt \mid \varepsilon \\ &expr \rightarrow b \end{aligned} \tag{3.9}$$

它的分析表见表 3.3。

$M[e_part, \textbf{else}]$ 条目包括 $e_part \rightarrow \textbf{else}\ stmt$ 和 $e_part \rightarrow \varepsilon$，后者是因为 $\varepsilon \in FIRST(e_part)$ 并且 $FOLLOW(e_part) = \{\textbf{else}, \$\}$。可以通过删去该条目中的 $e_part \rightarrow \varepsilon$ 来解决这种二义性，这个选择刚好满足 **else** 和最接近的 **then** 配对这个约定。 □

可以证明，一个文法的预测分析表没有多重定义的条目，当且仅当该文法是 LL(1)的。还可以证明，算法 3.2 为 LL(1)文法 G 产生的分析表能分析 $L(G)$ 的所有句子，也仅能分析 $L(G)$ 的句子。

剩下的一个问题是，当分析表有多重定义的条目时应该怎么办。一种办法是求助于文法变换，消除左递归和提取所有可能的左因子，以期得到的新文法的分析表没有多重定义的条目。遗憾的是，有些文法不论怎么变化也不能产生 LL(1)文法，文法(3.9)是一个这样的例子，它的语言没有 LL(1)文法。正如所看见的那样，让 $M[e_part, \textbf{else}] = \{e_part \rightarrow \textbf{else}\ stmt\}$，仍可以用预测分析器对(3.9)进行分析。但是，一般来说，没有一个普遍适用的规则来删除多重定义的条目，使其成为单值而不影响分析器可识别的语言。

表 3.3 文法(3.9)的预测分析表

非终结符	输入符号					
	other	*b*	**else**	**if**	**then**	$
stmt	*stmt* → **other**			*stmt* → **if**…		
e_part			*e_part* → **else** *stmt* *e_part* → ε			*e_part* → ε
expr		*expr* → *b*				

使用预测分析的主要困难在于为源语言写一个能构造出预测分析器的文法。虽然左递归的消除和提左因子是简单的，但它们使得结果文法难于理解而且不易于翻译。3.5 节介绍的 LR 分析器可以克服这些问题。

3.3.6 预测分析的错误恢复

在讨论预测分析的错误恢复前，先对编译器的错误处理作一个概述。